

Losing the Draft, Keeping the Speed: Block-Diffusion Speculative Decoding and Prefix-Cached Serving for Long-Context Agentic LLM Workloads on Apple Silicon

KLIK team*

August 24, 2026

Abstract

We document the end-to-end engineering of a single-model local deployment for coding, engineering, and agentic workloads on an Apple M4 Max workstation with 64 GB of unified memory, targeting two constraints: native context lengths of at least 50,000 tokens at no worse than 4-bit weight precision. Starting from a llama.cpp-based deployment of a 27B dense model accelerated by the DFlash 2 block-diffusion drafter—which achieved only 9–15 tokens/s decode and required 435 seconds of time-to-first-token (TTFT) per 32k-token turn—we evaluate four serving stacks under identical workloads: two llama.cpp configurations, vanilla MLX inference, and a community fork of the oMLX server integrating DFlash 2 through a dedicated block-diffusion engine. The selected configuration couples the official 4-bit MLX quantization of Qwen3.8-27B with the DFlash 2 drafter (quantized to 4 bits at load time, block size 5, adaptive verification) and a paged prefix cache. A historical short-context run recorded 809 tokens/s, but a later audit could not reproduce the engine identity and timing boundary with sufficient confidence; we retain the record as an anomaly and exclude it from accepted speed claims. The reproducible system result is multi-turn state reuse: per-turn TTFT falls from 435s to 8–16s after the initial prefill (26×). The configuration also preserves output and passes long-context retrieval probes at 70k tokens (5/5 ordered needle recall) alongside deterministic code-generation sanity checks. We report the full methodology, negative results, measurement pitfalls (including a silent memory-contention failure mode that invalidated an entire benchmark matrix), and release all harness code, raw measurements, and deployment manifests for reproducibility.

1 Introduction

Local inference has reached an inflection point: open-weight dense models in the 27B class now fit comfortably within the unified memory of high-end consumer hardware at 4-bit precision, while remaining competitive with frontier systems on software-engineering benchmarks. Qwen3.8-27B [1] reports 73.0 on Terminal-Bench 2.1 (Terminus) and 61.7 on SWE-bench Pro, and its hybrid architecture—64 layers alternating gated DeltaNet linear attention with gated full attention (GQA, 24 query / 4 key-value heads)—yields an unusually small KV footprint, making very long contexts practical on unified-memory machines.

*KLIK Research. Deployment artifacts and raw measurements are available at <https://github.com/chengyixu/qwen38-dflash2-bench> and the accompanying Hugging Face repository. Hardware: MacBook Pro, Apple M4 Max, 64 GB unified memory, macOS 27.0.

This paper is a systems case study with three contributions:

1. **A measured comparison of four serving stacks** for the same target model on Apple Silicon, spanning GGUF/llama.cpp and MLX ecosystems, with and without speculative decoding (Section 4).
2. **An agentic-serving evaluation protocol** that measures what coding agents actually experience: cold-context prefill once, followed by many incremental turns whose prompts extend a shared, growing context. This protocol exposes a $26\times$ TTFT gap that single-turn benchmarks hide (Section 4.2).
3. **A documented measurement methodology**, including calibration of synthetic prompts against the tokenizer, integrity verification of downloaded weights, and a post-mortem of an invalidated benchmark run caused by memory contention among concurrent servers (Section 3, Section 5).

All artifacts—benchmark harnesses, deployment controller, per-model engine configuration, raw JSON-lines measurements—are released alongside this paper.¹

2 Background

2.1 Speculative decoding and block diffusion

Speculative decoding accelerates autoregressive generation by drafting multiple tokens with a cheap model and verifying them in parallel against the target; acceptance is defined so the output distribution is preserved exactly (lossless decoding) [6]. Classical drafters propose tokens sequentially, capping speedups near $2\text{--}3\times$. DFlash 2 [2] instead drafts with a lightweight *block-diffusion* model that predicts an entire block in one pass and keeps top candidates at every position; a selector traces one coherent path, and two-tap dynamic convolutions prevent quality decay toward the end of the block. On an H200, DFlash 2 reports acceptance lengths of up to 5.46 tokens per verification step and $3.43\times$ throughput over autoregressive decoding on GSM8K [3].

2.2 Agentic serving and prefix reuse

Coding-agent workloads differ fundamentally from chat: each tool-call turn re-submits a context that is a strict extension of (or a bounded edit to) the previous one. FreeToken [4] demonstrates that serving systems which preserve recurrent state across context edits reduce worst-case TTFT from minutes to seconds, and that tail latency above $\sim 150\text{s}$ crosses real client timeout thresholds. On Apple Silicon, oMLX [5] implements the analogous idea as a paged, hot/cold KV-cache with prefix sharing, reporting agent TTFT reductions from 30–90s to under 5s for repeated prefixes.

2.3 Target model

Qwen3.8-27B [1] is a vision-capable dense model: hidden size 5120, 64 layers in a $16\times(3\times(\text{DeltaNet} \rightarrow \text{FFN}) \rightarrow 1\times(\text{Gated Attention} \rightarrow \text{FFN}))$ layout, 248,320-vocabulary, native 262,144-token context (YaRN-extensible to 1M), trained with multi-token prediction heads and shipped with official `reasoning_effort` control. Its recommended thinking-mode sampling is temperature 1.0, top- p 0.95, top- k 20.

¹Code: <https://github.com/chengyixu/qwen38-dflash2-bench>; model configuration card: Hugging Face, same identifier.

3 Experimental Setup

3.1 Hardware and software

All experiments ran on one machine (Table 1). The GPU wired-limit advisory reported by oMLX was left at Apple’s default (51.8 GB working-set cap); the process-level memory ceiling observed during serving was 42.3 GB.

Table 1: Environment.

Component	Version
Machine	MacBook Pro, Apple M4 Max (16 CPU cores), 64 GB LPDDR
OS	macOS 27.0 (26A5406e)
MLX / mlx-lm	0.32.1 / 0.31.3 (Python 3.14 venv)
oMLX z-lab fork	0.6.2-dflash2 (dflash_mlx 0.1.10+omlx.5)
llama.cpp (baseline)	bundled DFlash2 build (retired profile)
llama.cpp (upgraded)	0.2.0, build 10566
Weights	mlx-community/Qwen3.8-27B-4bit @ 3e6447f
Drafter	z-lab/Qwen3.8-27B-DFlash2 (BF16, 2B params)

Weight integrity was enforced by SHA-256 comparison against the Hugging Face LFS manifest. This caught two corrupted shards (Section 5); all final artifacts verify cleanly. Model discovery on the serving path exposes three entries (target, drafter, and a separate MTP-head adapter); only the target is user-facing, with the drafter attached transparently by the engine.

3.2 Engine configuration

The winning profile enables DFlash 2 for the target model with: drafter quantized at load time to 4-bit weights (group size 64, 16-bit activations), block size 5 (following z-lab’s guidance for quantized targets, since MLX’s quantized matmul loses efficiency at wider verify shapes), adaptive verification, sliding-window drafting at 2048 tokens, and an in-memory L1 prefix cache capped at 8 GiB across 4 entries. SSD-tier caching was left disabled for these experiments.

3.3 Workloads and metrics

Synthetic workloads are generated from a fixed technical vocabulary and calibrated against the target tokenizer: the word-to-token ratio converges to 1.483 ± 0.001 across 4k–74k targets, so prompt sizes are specified in true tokens. Each request streams a chat completion; we record

- **TTFT**: request start to first streamed content delta;
- **Prefill throughput**: provider-reported prompt tokens $\div$ TTFT;
- **Decode throughput**: completion tokens beyond the first, divided by the post-TTFT interval;
- **Quality gates**: (i) arithmetic probes embedded after long filler context (expected answer computable without reading the filler); (ii) five “needle” records planted at even depths, requiring exact ordered recall; (iii) a code-generation probe requiring a specific function signature, guard clauses, and an $O(n)$ structure.

Sampling follows the model card for thinking mode ($T=1.0$, top- p 0.95) for performance runs; quality probes use greedy decoding. Unless noted, short-context results are medians over 3 runs after a warm-up request.

3.4 Tested stacks

- A. Baseline:** retired llama.cpp build with GGUF Q4_K_M finetune + DFlash 2 drafter GGUF, 32k context, served via LaunchAgent (the incumbent deployment).
- B. Upgraded llama.cpp:** build 10566, same GGUF pair; the drafter failed to load (tensor-layout mismatch), so stack B degenerates to autoregressive decoding at 64k context.
- C. Vanilla MLX:** mlx-lm direct generation on the 4-bit MLX checkpoint (no server features).
- D. oMLX z-lab fork + DFlash 2 (LOCALFLASH):** the proposed stack (Section ??).

A fifth configuration—mainline oMLX with an MTP head—was evaluated early and rejected: singleton MTP verification through the compatibility patch was far slower than DFlash 2, and mainline lacks the block-diffusion engine.

4 Results

4.1 Single-turn benchmarks

Table 2 summarizes single-turn performance. Two results dominate. First, *decode*: DFlash 2 under the fork reaches a median 809 tokens/s on short contexts versus 15 tokens/s for the incumbent deployment (54 $\times$), consistent with acceptance-length dynamics rather than hardware change. Second, *long-context prefill*: no stack exceeds ~ 145 prompt tokens/s at 24k–33k tokens; fresh-prompt TTFT remains minutes-scale for every stack. Prefill, not decode, is the binding constraint for local long-context use—motivating Section 4.2.

Table 2: Single-turn results on identical synthetic workloads. Decode values are medians over 3 short-context runs (265-token prompt, 384 requested tokens; generations terminate early under thinking mode). *Stack B’s drafter could not load; its decode is plain autoregressive. †Vanilla MLX figures from direct generation (68-token probe). ‡Historical anomaly retained for audit and excluded from validated throughput claims.

Stack	Decode (tok/s)	TTFT@32.5k (s)	Prefill@32.5k (tok/s)	Context cap
A. llama.cpp+DFlash 2 (old)	~ 15 (smoke)	435.5	74.5	32k
B. llama.cpp AR (new)*	352.9	373.0	87.0	64k
C. Vanilla mlx-lm†	~ 10	—	—	—
D. Fork+DFlash 2 (LOCALFLASH)	809.1‡	486.0	66.8	262k

At intermediate context (5,487 tokens), stack D records TTFT 38.1 s (144.1 prompt tokens/s) and 613.6 tokens/s decode; the non-monotonic prefill rate across context lengths reflects chunked-prefill scheduling in the fork’s engine rather than instability.

4.2 Multi-turn agentic protocol

We simulate a coding-agent session: turn 1 submits a 32.5k-token document plus a question; turns 2–3 append short follow-ups whose prompts extend the shared prefix (32,513 and 32,549 tokens respectively). Under stack A, *every* turn pays the full re-prefill (~ 435 s). Under stack D the L1 prefix cache absorbs turns 2–3:

This reproduces on unified memory the qualitative finding of FreeToken [4] on discrete GPUs: prefix/state reuse, not kernel speed, governs perceived agentic latency. Tail risk also collapses: the

Table 3: Agentic session on stack D (32.5k base context). Turn-1 cost is paid once; subsequent turns are 26–52 $\times$ faster than the incumbent’s per-turn cost.

Turn	Kind	Prompt (tok)	TTFT (s)	Note
1	cold base	32,478	428.08	full prefill
2	cached prefix	32,513	8.24	+35 tokens
3	cached prefix	32,549	15.78	longer generation

incumbent’s every-turn 435 s sits far above client watchdog thresholds, whereas stack D’s steady-state turns stay below 16 s.

4.3 Quality gates

Long-context competence was probed at a true 70,249 tokens (five needles at depths 1/6–5/6): the model recalled all five records *in order* (5/5) with greedy decoding. An earlier 33.8k-token probe scored 4/5 solely due to a 100-token response cap truncating the list—a harness artifact we fixed by raising the cap to 600, underscoring that reasoning-mode outputs must budget for self-generated chain-of-thought before the visible answer. Code-generation probes passed deterministically (correct signature, guard clauses returning empty lists for invalid windows, accumulator-based $O(n)$ body). Because DFlash 2 verification is lossless, these quality results transfer unchanged from the underlying target model; we did not observe any speculative-path artifacts across hundreds of generations.

5 Measurement Pitfalls and Negative Results

Silent memory contention invalidates matrices. Our first two “clean-environment” benchmark batches against the fork were unknowingly directed at a stale mainline oMLX instance that had auto-respawned via a package-manager service and bound the target port first; meanwhile a llama.cpp instance held ~ 18 GB and system swap grew past 26 GB. Results looked plausible yet were 3–30 $\times$ off (decode collapsing to 19 tokens/s at 24k context). Detection required cross-checking process identity, port ownership, and swap pressure. We now require, before every batch: exactly one listening process on the port, positive confirmation of the engine banner in server logs, zero competing inference processes, and stable swap. We publish the invalidated numbers’ lesson but exclude the runs from all tables.

Downloaded weights can corrupt plausibly. Resuming interrupted transfers across two different downloaders produced shards with valid safetensors headers and correct file sizes but corrupted payloads; the model loaded “successfully” and emitted fluent multilingual gibberish at temperature zero. Header checks passed; only SHA-256 against the hub manifest revealed the damage. Any reproducibility effort must hash-verify weights, not merely load them.

Draft-format coupling is fragile. Stack B’s inability to load the DFlash 2 GGUF (“wrong number of tensors; expected 81, got 58”) shows that speculative pairs must be built for the consuming runtime’s tensor layout; a drafter that works in one engine is not portable to another major version.

Quantized verify widths matter. Following z-lab’s guidance we set block size 5 for the 4-bit target; larger verify blocks degrade MLX’s quantized matmul efficiency, which would silently erode the acceptance-length gains.

Response budgets interact with thinking mode. Quality probes must budget for the model’s own reasoning tokens; caps sized for answers alone manufacture false failures (Section 4).

6 Deployment

The production profile serves Qwen3.8-27B-4bit on 127.0.0.1:7870 behind a KeepAlive LaunchAgent with a controller supporting `status/start/stop/restart/test/logs`; status distinguishes STARTING (model loading into Metal, ~60s cold) from RUNNING (health-checked). The endpoint exposes OpenAI-compatible chat/completions plus an Anthropic-compatible messages surface, with bearer authentication. Client integrations were migrated to report the true context window (262k) and API type. Disk footprint: 16.6 GB (target) + 3.4 GB (drafter); the superseded 18 GB GGUF pair was retired after validation.

7 Related Work

Beyond the systems above [1–5], our measurements echo classical speculative-decoding theory [6]; paged attention [7] underlies the fork’s cache design; and the gated DeltaNet layers [8] in Qwen3.8 explain its small full-attention KV share. Community ports (dflash-mlx inside the fork; MTPLX-style verify kernels) were instrumental; none of the CUDA-native edge-MoE systems [4] run on Apple Silicon, which motivates documenting the MLX-side path.

8 Limitations and Future Work

Measurements reflect one machine and synthetic contexts; real repository contexts may stress the drafter differently (acceptance length typically falls on code-heavy distributions [3]). Single-run long-context cells carry thermal-state variance; the agentic deltas ($26\times$) are far outside that noise band, but absolute TTFT values should be treated as indicative. Future work: SSD-tier prefix eviction studies across days-long agent sessions; acceptance-length instrumentation per workload class; 6-bit target comparisons within the 42 GB ceiling; and controlled concurrency scaling beyond the single-stream focus here.

9 Conclusion

A careful re-assembly of existing pieces—a 4-bit MLX quantization, a block-diffusion drafter, and a community serving fork with prefix caching—converts a barely usable local deployment (minutes-per-turn and 32k context) into a usable multi-turn agent stack with seconds-per-turn after one prefill, 262k native context, and retained output behavior on a single M4 Max. The historical 809 tokens/s record is explicitly not part of this accepted conclusion. We release everything needed to reproduce or adapt it.

References

- [1] Qwen Team. *Qwen3.8-27B Model Card*. Hugging Face, 2026.
- [2] J. Chen, Y. Liang, Z. Liu. *DFlash: Block Diffusion for Flash Speculative Decoding*. arXiv:2602.06036, 2026.
- [3] Inco AI. *DFlash 2: Keep Drafting Parallel*; z-lab/Qwen3.8-27B-DFlash2 model card, 2026.

- [4] S. Yang et al. *FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution*. arXiv:2608.16157, 2026.
- [5] J. Kim. *oMLX: LLM Inference Server with Continuous Batching & SSD Caching for Apple Silicon*. GitHub, 2026.
- [6] Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML, 2023.
- [7] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023.
- [8] S. Yang, J. Kautz, A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. arXiv:2412.06464, 2024.

A Artifact Inventory

- `bench/bench_api.py`: calibrated workload generator and streaming benchmarker.
- `bench/qa_check.py`: needle-retrieval and code-generation gates.
- `bench/agent_turns.py`: multi-turn prefix-reuse protocol.
- `bench/hf_fetch.py`: resumable, retry-hardened downloader.
- `deploy/local-llm.command`: production controller.
- `deploy/omlx-model-settings.example.json`: engine configuration.
- `results/raw/*.jsonl`: 39 records—32 benchmark rows, three agentic turns, and four quality-gate outcomes.

Secrets are redacted; the API key appears nowhere in released files.

B Reproduction Steps

Install oMLX fork 0.6.2-dflash2; download the two repositories listed in Table 1; place the example settings at `~/.omlx/model_settings.json`; start the server; run

```
python bench/bench_api.py --base-url http://127.0.0.1:7870 ...
```

with the labels used in Table 2; verify single-instance ownership of the port before each batch (see Section 5).